Threading and Task



Soyeb Ghachi

Technical Trainer | Mentor |SME

Overview



Threading and Task

-What is Threading (System.Threading)

-What is Task (System.Threading.Task)?

-async and await keyword

System.Threading.Tasks

What is Thread (System.Threading)?

In C#, a thread is the smallest unit of execution within a process. It is a lightweight, independent path of execution that allows concurrent or parallel execution of code. Threads enable the execution of multiple tasks or operations simultaneously, enhancing the performance and responsiveness of an application.

- **Concurrent execution:** Threads allow multiple pieces of code to be executed concurrently, enabling tasks to run in parallel and utilize the available processing power efficiently.
- **Multithreading:** C# provides built-in support for multithreading, which involves creating and managing multiple threads within an application. Multithreading enables concurrent execution of tasks, such as performing calculations, handling I/O operations, or responding to user interactions, without blocking the main thread.

Example:

```
using System;
using System.Threading;

public class ThreadExample
{
    // Method to be executed in the thread
    public static void ThreadMethod()
    {
        Console.WriteLine("Thread is running");
        // Simulating some work
        Thread.Sleep(2000);
        Console.WriteLine("Thread has finished");
    }
```

```
 public static void Main(string[] args)
    {
        // Creating a new thread and providing the method
to be executed
        Thread thread = new Thread(ThreadMethod);

        // Starting the thread
        thread.Start();

        // Main thread continues execution while the new
thread is running concurrently

        Console.WriteLine("Main thread is doing some other
work");

        // Waiting for the new thread to complete
        thread.Join();

        Console.WriteLine("Main thread has finished");
    }
}
```

What is Task (System.Thread.Task)?

- Provides types that simplify the work of writing concurrent and asynchronous code.
- Task is a lightweight object for managing a parallelizable unit of work. It can be used whenever you want to execute something in parallel.
- Parallel means the work is spread across multiple processors to maximize computational speed.
- Tasks are tuned for leveraging multi-cores processors.
- The Thread class is used for creating and manipulating a thread in Windows.
- A Task represents some asynchronous operation and is part of the Task Parallel Library, a set of APIs for running tasks asynchronously and in parallel

- The task can return a result. There is no direct mechanism to return the result from a thread.
- A Task is a higher level concept than Thread.
- The main types are Task which represents an asynchronous operation that can be waited on and cancelled.
- Task<TResult>, which is a task that can return a value.

Example:

```
using System;
using System.Threading;
using System.Threading.Tasks;

class Program
{
    static void Main()
    {
        // Create multiple tasks to execute concurrently
        Task task1 = Task.Run(() =>
        {
            Console.WriteLine("Task 1 is executing...");
            Thread.Sleep(2000);
            Console.WriteLine("Task 1 completed.");
        });
```

```
Task task2 = Task.Run(() =>
        {
            Console.WriteLine("Task 2 is executing...");
            Thread.Sleep(1500);
            Console.WriteLine("Task 2 completed.");
        });


        // Wait for all tasks to complete
        Task.WaitAll(task1, task2);

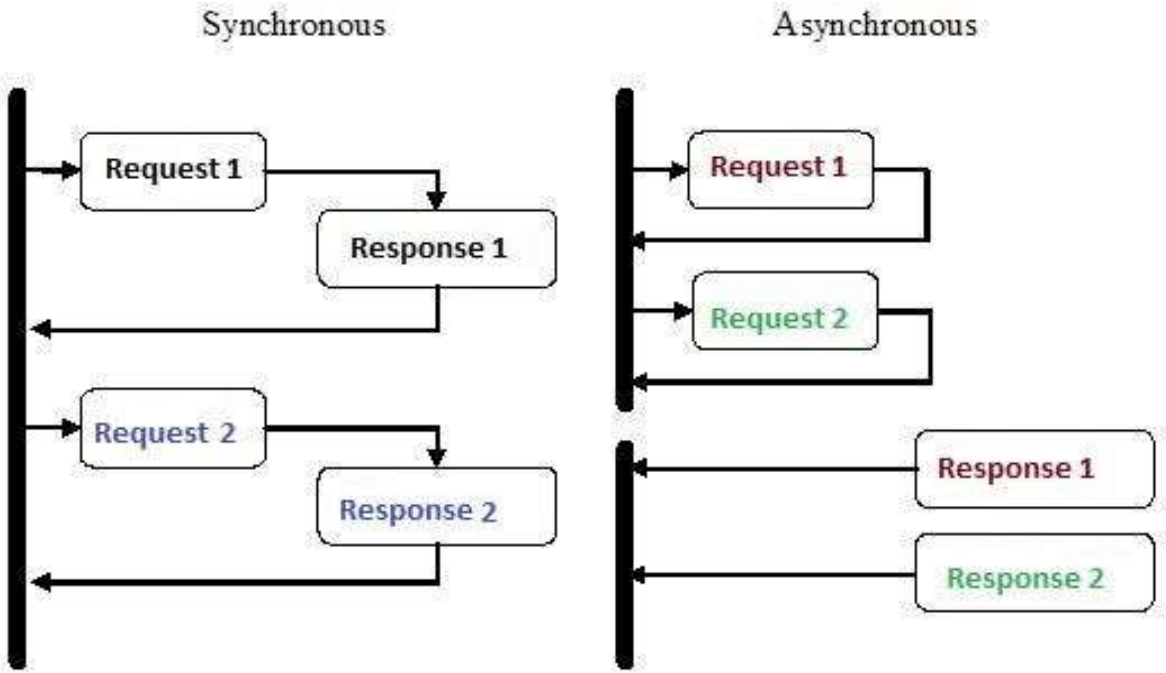
        Console.WriteLine("All tasks completed.");
    }
}
```

Asynchronous programming with async and await keyword

- The Task asynchronous programming model (TAP) provides an abstraction over asynchronous code.
- You write code as a sequence of statements, just like always. You can read that code as though each statement completes before the next begins.
- When we are dealing with UI, and on button click, we use a long-running method like reading a large file or something else which will take a long time, in that case, the entire application must wait to complete the whole task.
- In other words, if any process is blocked in a synchronous application, the whole application gets blocked, and our application stops responding until the whole task completes.

- Asynchronous programming is very helpful in this condition. By using Asynchronous programming, the Application can continue with the other work that does not depend on the completion of the entire task.

- We will get all the benefits of traditional Asynchronous programming with much less effort with the help of async and await keywords.



Example:

```
using System;
using System.Threading.Tasks;

class Program
{
    static async Task Main()
    {
        Console.WriteLine("Before calling the async method.");

        await DoSomethingAsync();

        Console.WriteLine("After calling the async method.");
    }
```

```
static async Task DoSomethingAsync()
    {
        Console.WriteLine("Async method started.");

        // Simulate an asynchronous operation
        await Task.Delay(2000);

        Console.WriteLine("Async method completed.");
    }
}
```

Output:

```
Before calling the async method.
Async method started.
Async method completed.
After calling the async method.
```

References

https://learn.microsoft.com/en-us/dotnet/api/system.threading.tasks.task?view=net-7.0

https://dotnettutorials.net/lesson/asynchronous-programming-in-csharp/

https://www.c-sharpcorner.com/article/task-and-thread-in-c-sharp/

https://www.tutlane.com/tutorial/csharp/csharp-task

https://www.tutorialsteacher.com/articles/asynchronous-programming-with-async-await-task-csharp

https://www.javatpoint.com/task-vs-thread-c-sharp

https://www.linkedin.com/pulse/c-threading-tasks-async-code-synchronization-part-2-meikopoulos